1) **Q1.py** Question 1 — Movie Ratings (Lists and Strings)

Approach :

1. **4 functions created as instructed**
2. **parse_ratings (data: str)**
   Used **split()** to separate the ratings string into individual movie-rating entries.
   Used **strip()** to remove extra spaces around movie names and ratings while parsing the ratings data
   Converted ratings to integers and stored them as **(movie, rating)** tuples in a list.
3. **average_rating(ratings, title)**
   Used **list comprehensions**, **sum()**, and **len()** to calculate the average rating for each movie.
4. **best_movie(ratings)**
   Used a **set** to collect unique movie titles and identify the best-rated movie.
5. **rating_counts(ratings)**
   Used a **dictionary** to count how many ratings each movie received.

Demo Output:

```
print("Parsed ratings:", ratings)
print("Average rating for Dune:", average_rating(ratings, "Dune"))
print("Average rating for Barbie:", average_rating(ratings, "Barbie"))
print("Average rating for Oppenheimer:", average_rating(ratings, "Oppenheimer"))
print("Average rating for None:", average_rating(ratings, "None"))
print("Best movie:", best_movie(ratings))
print("Rating counts:", rating_counts(ratings))
```

```
Parsed ratings: [('Dune', 8), ('Dune', 9), ('Barbie', 7), ('Dune', 10), ('Barbie', 9), ('Oppenheimer', 9), ('Barbie', 6)]
Average rating for Dune: 9.0
Average rating for Barbie: 7.3
Average rating for Oppenheimer: 9.0
Average rating for None: 0.0
Best movie: Oppenheimer
Rating counts: {'Dune': 3, 'Barbie': 3, 'Oppenheimer': 1}
```

Problem ran into: None

2) **Q2.py** Gradebook (Dictionaries and Sets)

Approach :

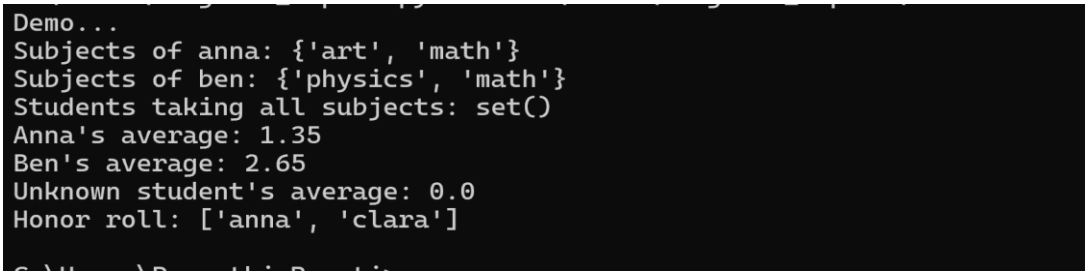
- Used grades.items() to iterate through subjects and student records.
- **subjects_of(student)**

Used a set and add() to find all subjects taken by a student.

- **takes_all(grades)**
  Used list comprehension and set.intersection() to find students enrolled in every subject.
- **student_average(grades, student)**
  Used a list, append(), sum(), and len() to calculate a student's average grade.
- **honor_roll(grades, limit=1.5)**
  Used a set and update() to collect all unique students across subjects.
  Used sorted() to return the honor roll list in alphabetical order.

Demo Output:

```
print("Demo... ")
print("Subjects of anna:", subjects_of("anna"))
print("Subjects of ben:", subjects_of("ben"))
print("Students taking all subjects:", takes_all(grades))
print("Anna's average:", student_average(grades, "anna"))
print("Ben's average:", student_average(grades, "ben"))
print("Unknown student's average:", student_average(grades, "eva"))
print("Honor roll:", honor_roll(grades))
```



Problem ran into: None

3) **Q3.py** Recipe Scaler (Functions)

Approach :

- Create **scale_recipe(name, servings, *ingredients, unit="g", **options)**
- Used *ingredients to accept a variable number of ingredients as tuples.
- Used **options to accept optional cooking notes such as time and style.
- Used a for loop to iterate through ingredients and scale each quantity based on the number of servings.
- Used a dictionary to store the scaled ingredient amounts.

- Used an if statement to validate that servings are at least 1 and handle invalid input.
- Used options.items() to display additional cooking notes when provided.

Demo Output:

```
print("Demo.... ")

# Simple recipe
print(scale_recipe("chocolate cake",4,("All purpose", 100),("Milk", 200),("Sugar", 20),("coco powder", 20)))

# Different unit
print(scale_recipe("Lemonade", 3,("Water", 250),("Lemon Juice", 50),unit="ml"))

# Several options
print(scale_recipe("Biryani",2,("Rice", 300),("Masala", 150),("Ghee", 100),("Onions", 100),time="60min",Style="Dum"))

# Invalid servings
print(scale_recipe("dessert",0,("Flour", 200),("Sugar", 100)))
```

```
Demo....

Recipe: chocolate cake
Servings: 4
Shopping List:
- All purpose: 400 g
- Milk: 800 g
- Sugar: 80 g
- coco powder: 80 g
{'All purpose': 400, 'Milk': 800, 'Sugar': 80, 'coco powder': 80}

Recipe: Lemonade
Servings: 3
Shopping List:
- Water: 750 ml
- Lemon Juice: 150 ml
{'Water': 750, 'Lemon Juice': 150}

Recipe: Biryani
Servings: 2
Shopping List:
- Rice: 600 g
- Masala: 300 g
- Ghee: 200 g
- Onions: 200 g
Cooking Notes:
  time: 60min
  Style: Dum
{'Rice': 600, 'Masala': 300, 'Ghee': 200, 'Onions': 200}

Recipe: dessert
Servings: 0
Shopping List:
Error - servings must be at least 1 : dessert
0
```

Problem ran into: printing ingredient formatting error and got them fixed.

Had to move the check condition on serving < 1 after the print recipe, otherwise its getting terminated without displaying the receipe details where serving <1

4) **Q4.py** Library (Classes and Inheritance)

Approach :

- Created Super class **Book** that prints in nice required format and also checks for dups and calculates the age of book,
- Created subclass **EBook** that inherits super class Book which has additional attribute size_mb to calculate download time in seconds
- created class **Library** to store the books list, add books, find author , no.of books, oldest book
- Used super() to call the parent class constructor from the child class.
- Used (__str__, __eq__, and __len__) methods for printing in required format , duplicate checking, and counting books.
- Used a list to store books in the library class and a list comprehension to find books by a specific author.
- Used a for loop and conditional statements to identify the oldest book.
- Used datetime.now().year to calculate the current age of a book dynamically. For this , I have imported datetime from library

Demo Output:

```
print("Demo.... ")

print("\nAdding books..... ")

b1 = Book("1984", "George Orwell", 1949)
b2 = Book("Core Python Programming", "R. Nageswara Rao", 2007)
b3 = Book("Robotics, Vision and Control", "Peter Corke", 2011)
b4 = Book("Coming Up for Air", "George Orwell", 1939)
e1 = EBook("Python Basics", "John Smith", 2020, 25)
e2 = EBook("Data Science", "Jane Doe", 2022, 40)
duplicate = Book("1984", "George Orwell", 1950)

lib = Library()
```

```
lib.add(b1)
lib.add(b2)
lib.add(b3)
lib.add(b4)
lib.add(e1)
lib.add(e2)
lib.add(duplicate)  # this will be ignored while adding

print("\nTesting..... ")
print(f"\nAge of {b1} : ", b1.age(datetime.now().year))
print(f"\nDownload time of {e1} : ", e1.download_seconds(20), "seconds")
print("\nBooks by George Orwell:")
for book in lib.find_by_author("George Orwell"):
   print(book)

print("\nOldest book : " ,lib.oldest())
print("\nNumber of books in library : ", len(lib))
print("\nAll books in library :")
for book in lib.books:
   print(book)
```

```
Demo....

Adding books.........
Book added: 1984 by George Orwell (1949)
Book added: Core Python Programming by R. Nageswara Rao (2007)
Book added: Robotics, Vision and Control by Peter Corke (2011)
Book added: Coming Up for Air by George Orwell (1939)
Book added: Python Basics by John Smith (2020) [25 MB]
Book added: Data Science by Jane Doe (2022) [40 MB]
Duplicate book ignored: 1984 by George Orwell (1950)

Testing.........

Age of 1984 by George Orwell (1949) :  77

Download time of Python Basics by John Smith (2020) [25 MB] :  10.0 seconds

Books by George Orwell:
1984 by George Orwell (1949)
Coming Up for Air by George Orwell (1939)

Oldest book :  Coming Up for Air by George Orwell (1939)

Number of books in library :  6

All books in library :
1984 by George Orwell (1949)
Core Python Programming by R. Nageswara Rao (2007)
Robotics, Vision and Control by Peter Corke (2011)
Coming Up for Air by George Orwell (1939)
Python Basics by John Smith (2020) [25 MB]
Data Science by Jane Doe (2022) [40 MB]
```

Problem ran into :

The duplicate book check in class Library below, printing both messages if we don't specify the return . Without 'return' Python is continuing executing the remaining part.

```
def add(self, book):
    if book in self.books:
        print(f"Duplicate book ignored: {book}")
        return          -----  added this to avoid printing both messages.
    else:
        self.books.append(book)
    print(f"Book added: {book}")
```